

**SIMATS ENGINEERING  
ASSESSMENT TOOL 1**

**COURSE NAME: COMPUTER NETWORKS  
COURSE CODE: CSA07**

---

**COURSE OUTCOME COVERED**

**CO4:**

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards  
(BL2,BL6)

Assessment Tool Weightage: 40%

---

**QUESTIONS: 5**

**Total Marks:25**

Annexure A

Coding challenge

---

**Code of Conduct:**

I J. Kirthiga / 1924 21265 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.

J. Kirthiga  
Signature of the student:

20  
20

R. L. P.

## Annexure A

### Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given  $n$  hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits ( $\leq 100$  m) and False with an error message otherwise. Include at least 3 test cases.
3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma_log.txt`.
4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class `StarSwitch` with methods `add_port(host_id)`, `remove_port(host_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

### RUBRICS FOR CALCULATION

| Criteria              | Weightage | Exemplary (4)                                                           | Proficient (3)                                         | Developing (2)                                  | Beginning (1)                     |
|-----------------------|-----------|-------------------------------------------------------------------------|--------------------------------------------------------|-------------------------------------------------|-----------------------------------|
| Problem Understanding | 15%       | Fully understands the problem and requirements; no clarification needed | Understands most requirements with minor clarification | Partial understanding; misses some requirements | Poor understanding of the problem |
| Algorithm Design      | 20%       | Efficient, optimal, and well-structured algorithm                       | Correct algorithm with minor inefficiencies            | Basic approach but not optimal                  | Incorrect or no clear algorithm   |

## 1. Star Topology Diagram Generator

### CODE:

```
def generate_star_topology(n, switch_name="Switch-1"):
    print("\nStar Topology Diagram:\n")

    for i in range(1, n + 1):
        length = 10 * i # example cable length
        print(f"Host-{i} ---({length}m Cat6 UTP)---> {switch_name}")
```

### Output:

```
>>> | Enter help below or click help above for more
    | _____
    |
    | Star Topology Diagram:
    |
    | Host-1 --- (10m Cat6 UTP) ---> Switch-1
    | Host-2 --- (20m Cat6 UTP) ---> Switch-1
    | Host-3 --- (30m Cat6 UTP) ---> Switch-1
    | Host-4 --- (40m Cat6 UTP) ---> Switch-1
    |
    | >>>
```

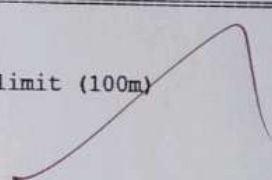
## 2. UTP Segment Validation Function

CODE:

```
def validate_segment(length_m):  
    if length_m <= 100:  
        return True  
    else:  
        print(f"Error: Length {length_m}m exceeds IEEE 802.3 limit (100m)")  
        return False
```

Output:

```
True  
True  
Error: Length 120m exceeds IEEE 802.3 limit (100m)  
False  
>>> |
```



### 3. CSMA/CD Simulation (4 Hosts)

CODE:

```
import random
import time
def csma_cd_simulation():
    hosts = ["H1", "H2", "H3", "H4"]

    # open file to store logs
    log_file = open("csma_log.txt", "w")

    for attempt in range(5):
        sender1, sender2 = random.sample(hosts, 2)
        collision = random.choice([True, False])
        if collision:
            backoff = random.randint(1, 10)
            log = f"Collision detected between {sender1} and {sender2}. Backoff: {backoff} slots\n"
            print(log.strip())
            log_file.write(log)
            time.sleep(0.5)
        else:
            log = f"{sender1} transmitted successfully.\n"
            print(log.strip())
            log_file.write(log)

    log_file.close()

csma_cd_simulation()
```

output:

>>>

H2 transmitted successfully.

H4 transmitted successfully.

H3 transmitted successfully.

Collision detected between H3 and H1. Backoff: 8 slots

H2 transmitted successfully.

>>>

RESTART:



#### 4. Client-Server (Star Switch Simulation using sockets)

##### CODE:

```
import socket
import threading
import time

HOST = '127.0.0.1'
PORT = 5000

mac_table = {} # MAC → connection
def handle_client(conn, addr):
    global mac_table
    while True:
        try:
            data = conn.recv(1024).decode()
            if not data:
                break
            src_mac, dest_mac, message = data.split('|')
            # Learn source MAC
            mac_table[src_mac] = conn

            print(f"\nFrame: {src_mac} → {dest_mac}")
            print(f"Message: {message}")

            # Forwarding
            if dest_mac in mac_table:
                print(f"Forwarding to {dest_mac}")
                mac_table[dest_mac].send(data.encode())
```

```
    else:
        print(" Broadcasting (unknown destination)")
        for mac, client in mac_table.items():
            if client != conn:
                client.send(data.encode())

    # Print MAC Table
    print("\n MAC Address Table:")
    for mac in mac_table:
        print(f" {mac}")

except:
    break
conn.close()
def start_switch():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen()
    print("Switch started...\n")
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr),
            daemon=True).start()
def start_host(mac_address, messages):
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((HOST, PORT))
    def receive():
        while True:
            data = client.recv(1024).decode()
```

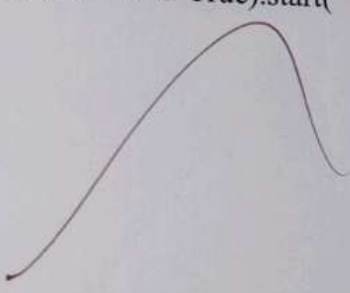
```
src, dest, msg = data.split("|")
if dest == mac_address:
    print(f"\n Host {mac_address} received from {src}: {msg}")
except:
    break

threading.Thread(target=receive, daemon=True).start()
Send messages automatically
for dest_mac, msg in messages:
    time.sleep(2)
    frame = f"{mac_address}|{dest_mac}|{msg}"
    print(f"\n Host {mac_address} sending → {dest_mac}: {msg}")
    client.send(frame.encode())

threading.Thread(target=start_switch, daemon=True).start(
time.sleep(1)  hostA_msgs = [
    ("B1", "Hello from A"),
    ("B1", "How are you?")
hostB_msgs = [
    ("A1", "Hi A, I'm B"),
    ("A1", "I'm good!")
    threading.Thread(target=start_host, args=("A1", hostA_msgs),
daemon=True).start()

    threading.Thread(target=start_host, args=("B1", hostB_msgs),
daemon=True).start()

while True:
    time.sleep(1)
```



## Output:

```
WINBOX-1717
File Edit View Settings Options Window Help
Host A1 sending - R1: Hello from A

From: R1 - R1
From: R1 - R1
Message: R1 A. I'm good Message: Hello from A
Forwarding to R1 Forwarding to R1

Host A1 received from R1: R1 A. I'm good
Host A1 received from R1: Hello from A
Host A1 received from R1: Hello from A
Host A1 received from R1: Hello from A

R1 R1
A1 R1

Host R1 sending - A1: I'm good
Host A1 sending - R1: How are you?

From: R1 - R1
From: A1 - R1
Message: I'm good Message: How are you?
Forwarding to A1 Forwarding to R1

Host R1 received from R1: I'm good
Host R1 received from R1: How are you?
Host R1 received from R1: How are you?
Host R1 received from R1: How are you?

R1 R1
A1 R1
```

## \* StarSwitch: Simple Broadcast-Based Communication in Star Topology:

CODE:

```
class StarSwitch:
    def __init__(self):
        self.ports = {} # host_id → list (simulated message log)

    def add_port(self, host_id):
        if host_id not in self.ports:
            self.ports[host_id] = []
            print(f"Port added for Host {host_id}")
        else:
            print(f"Host {host_id} already exists")

    def remove_port(self, host_id):
        if host_id in self.ports:
            del self.ports[host_id]
            print(f"Port removed for Host {host_id}")
        else:
            print(f"Host {host_id} not found")

    def broadcast(self, frame):
        src = frame["src"]
        data = frame["data"]
        print(f"\nBroadcasting frame from {src}: {data}")
        for host_id in self.ports:
            if host_id != src:
                # Deliver frame
                self.ports[host_id].append(data)
                print(f"Delivered to Host {host_id}")
```

```

switch = StarSwitch()
switch.add_port("A")
switch.add_port("B")
switch.add_port("C")
frame1 = {"src": "A", "data": "Hello Everyone"}
switch.broadcast(frame1)
frame2 = {"src": "B", "data": "Hi A and C"}
switch.broadcast(frame2)
switch.remove_port("C")
frame3 = {"src": "A", "data": "Only B should get this"}
switch.broadcast(frame3)
print("\n Host Message Logs:")
for host, messages in switch.ports.items():
    print(f'{host}: {messages}')

```

output:

```

RESTART: C:/Users/Arith/5.Py
Port added for Host A
Port added for Host B
Port added for Host C
Broadcasting frame from A: Hello Everyone
Delivered to Host B
Delivered to Host C
Broadcasting frame from B: Hi A and C
Delivered to Host A
Delivered to Host C
Port removed for Host C
Broadcasting frame from A: Only B should get this
Delivered to Host B
Host Message Logs:
A: ['Hi A and C']
B: ['Hello Everyone', 'Only B should get this']
>>

```